Deployment Guide

ResolveFlow is deployed on: - **Frontend:** Vercel (React app) - **Backend:** Render (FastAPI) - **Database:** Postgres (managed)

Current URLs

- **Frontend:** https://resolveflow-web-officialbidishas-projects.vercel.app
- **API:** https://resolveflow-1h99.onrender.com/api
- **Health Check:** https://resolveflow-1h99.onrender.com/api/health

Auto-Deploy on Push

The `.github/workflows/deploy.yml` workflow automatically: 1. Runs safety-gate evals on every push to main 2. Deploys frontend to Vercel (if evals pass) 3. Deploys backend to Render (if evals pass)

Flow:

```
Push to main
    ↓
Run eval.harness (12/12 safety gates)
    ↓
Deploy frontend + backend (if passing)
    ↓
Live at URLs above
```

Setup Requirements

GitHub Secrets (for CI/CD)

Set these in GitHub Settings → Secrets and variables → Actions:

For Vercel: - `VERCEL_TOKEN` — Personal access token (get from Vercel dashboard) - `VERCEL_ORG_ID` — Your Vercel org ID - `VERCEL_PROJECT_ID` — Your Vercel project ID

For Render: - `RENDER_SERVICE_ID` — Your Render service ID - `RENDER_API_KEY` — Your Render API key

For Evals: - `OPENAI_API_KEY` — Your OpenAI key (already set) - `PINECONE_API_KEY` — Your Pinecone key (already set)

Environment Variables

Vercel (Frontend)

```
VITE_API_URL=https://resolveflow-1h99.onrender.com
```

Set in Vercel Project Settings → Environment Variables

Render (Backend)

```
DATABASE_URL=postgresql://...    # Postgres connection string
OPENAI_API_KEY=sk-...            # OpenAI key
PINECONE_API_KEY=...             # Pinecone key
GITHUB_OAUTH_CLIENT_ID=...       # GitHub OAuth app ID
GITHUB_OAUTH_CLIENT_SECRET=...   # GitHub OAuth secret
FRONTEND_URL=https://resolveflow-web-officialbidishas-projects.vercel.app
```

Set in Render Dashboard → Environment

Manual Deployment

Deploy Frontend (Vercel)

```
# One-time setup
npm install -g vercel

# Deploy
vercel --prod

# Or use the Vercel dashboard to trigger deploys
```

Deploy Backend (Render)

```
# The render.yaml config handles everything
# Push to main → Render auto-deploys

# Manual trigger via Render dashboard:
# 1. Go to https://dashboard.render.com
# 2. Select your service
# 3. Click "Manual Deploy" → "Deploy latest commit"
```

Monitoring Deployments

Check Vercel Deployment

```
vercel logs --prod
# Or view in Vercel dashboard
```

Check Render Deployment

```
# View logs in Render dashboard
# Or check health:
curl https://resolveflow-1h99.onrender.com/api/health
```

Check Database

```
# Connect to Postgres
psql $DATABASE_URL
```

```
# Verify new tables exist (from feedback system)
\dt feedback
SELECT COUNT(*) FROM feedback;
```

Rollback

If a deployment breaks production:

Vercel

1. Go to Vercel dashboard → Deployments
2. Find the last working deployment
3. Click "Rollback" → Confirm

Render

1. Go to Render dashboard → your service → Events
2. Find the last successful deploy
3. Click the deployment ID → "Redeploy"

Pre-Deployment Checklist

Before pushing to main:

- [ ] Run local evals: `python -m eval.harness`
- [ ] Run capability evals: `python -m eval.capability_harness`
- [ ] Test frontend locally: `cd frontend && npm run dev`
- [ ] Test backend locally: `uv run uvicorn app.main:app --reload`
- [ ] Check for uncommitted changes: `git status`
- [ ] Pull latest: `git pull origin main`

Performance & Scaling

Render Free Tier

Current setup runs on Render's free tier with: - **Limits:** 0.5 GB RAM, auto-stops after 15 min inactivity - **Cold starts:** ~30s on first request after inactivity - **Workaround:** The `keep-warm.yml` workflow pings the API every 15 min

Scaling Up

To move to production capacity:

1. **Upgrade Render Plan** - Go to Settings → Instance Type - Choose Starter ($7/month) or higher

2. **Database** - Currently Postgres (managed) - Upgrade to dedicated instance if needed

3. **Frontend Caching** - Vercel has CDN by default - Enable aggressive caching in `vercel.json` for faster loads

Database Migrations

When schema changes land (like the new `feedback` table):

1. **Local:** Migrations run on `app.setup()`

2. **Prod:** Migrations run on Render startup

To force a migration in production:

```
# Connect to production Postgres
psql $DATABASE_URL

# Run SQL manually if needed
CREATE TABLE IF NOT EXISTS feedback (
  id SERIAL PRIMARY KEY,
  ...
);
```

Troubleshooting

"Deploy failed" in GitHub Actions

```
# Check what failed
git log --oneline -5
# Was it eval failures or deploy failure?

# Run evals locally
python -m eval.harness

# If evals fail, fix the code before pushing again
```

Frontend showing "API unreachable"

```
# Check if backend is up
curl https://resolveflow-1h99.onrender.com/api/health

# If backend is down:
# 1. Check Render dashboard for errors
# 2. Verify DATABASE_URL is set in Render
# 3. Check Postgres connection
```

Slow cold starts

Backend is on free tier and gets cold-stopped after 15 min idle.

```
# The keep-warm workflow prevents this
# Check that .github/workflows/keep-warm.yml is running:
# Go to GitHub → Actions → "Keep Render warm"

# If still slow, upgrade to paid Render plan
```

Cost

Current estimated monthly cost: - Vercel: $0 (free tier, up to 100 deployments/month) - Render: $0 (free tier with cold-start penalty) - Postgres: $0-50 (managed, depends on data size and connections) - OpenAI: ~$5-10 (based on

usage) - Pinecone: $0 (free tier up to 1M vectors)

To reduce costs: - Keep using free tiers - Use scheduled evals instead of per-commit - Batch inference jobs during off-peak hours

To scale up to production, budget ~$50-100/month total.